Python Secrets and Advanced Concepts

A collection of useful Python tips and explanations for more advanced concepts.

`__str__` vs. `__repr__`: How `print()` Behaves

A common point of confusion is how Python decides to represent an object as a string. The key is to understand the difference between `__str__` and `__repr__` and how `print()` uses them.

- `print(object)` will try to use the `__str__()` method of the object. This is meant to be a user-friendly, readable representation.
- If `__str__()` is not defined, it falls back to `__repr__()`.
- When you print a container like a list (`print([object])`), the list's `__repr__` method is called, which in turn calls the `__repr__()` method of each object inside it. `__repr__()` is meant to be an unambiguous, developer-friendly representation that could ideally be used to recreate the object.

Example

```
class MyObject:
    def __init__(self, value):
        self.value = value

    def __str__(self):
        return f"A user-friendly string: {self.value}"

    def __repr__(self):
        return f"MyObject(value={self.value!r})"

obj = MyObject(10)

# print(object) uses __str__
print(obj)

# print([object]) uses __repr__ for the object inside the list
print([obj])
```

Output:

```
A user-friendly string: 10
[MyObject(value=10)]
```

Python Async Await Generators Coroutine

```
Concept                          await?              next()?
Notes
-------------                   -------------       -------------
------------

1. def + yield                      
```

```
Regular generators

2. async def + await                □                    □
Regular coroutines

3. async def + yield                □                    □ via async for
Async generators

4. __await__() object               □                    □ if it’s a generator
Custom awaitables
```

□ 1. def + yield

A regular generator (synchronous). You can next() it, but you can’t await it.

```
def gen():
    yield "hello"
    yield "world"

g = gen()
print(next(g))  # □ "hello"
print(next(g))  # □ "world"

# await g  □ TypeError: cannot 'await' a generator object
```

□ 2. async def + await

A coroutine function. You must use await, but you can’t next() it.

```
import asyncio

async def say_hello():
    return "Hello Async"

async def main():
    result = await say_hello()  # □ works
    print(result)

asyncio.run(main())

# say_hello() is a coroutine
# next(say_hello())  □ TypeError
```

□ 3. async def + yield

This defines an async generator. You can’t await it, but you can iterate with async for.

```
import asyncio

async def async_gen():
    for i in range(3):
        yield i  # □ allowed in async generators
```

```
async def main():
    async for val in async_gen():  # ✅ works
        print(val)

asyncio.run(main())

# await async_gen() ❌ TypeError: can't be awaited
# next(async_gen()) ❌ TypeError
```

❌ 4. Object with **await**()

Custom object that is awaitable. This can be await-ed and sometimes also next()-ed.

```
class CustomAwaitable:
    def __await__(self):
        print("Inside __await__")
        yield  # pause here
        return "Done!"

async def main():
    result = await CustomAwaitable()  # ✅ Yes, works
    print(result)

asyncio.run(main())

# Technically, you could also do:
g = CustomAwaitable().__await__()
print(next(g))  # ✅ works because __await__ returns a generator
```

A coroutine is a special kind of function in Python that can: • pause its execution at a certain point, • wait for input or other events, • and then resume from where it left off.

They are used to write non-blocking, asynchronous, and cooperative multitasking code in a clean and readable way.

✅

🧠 Simple Definition:

A coroutine is like a function that can pause, resume, and receive values, making it perfect for concurrent programming without threads.

🔁 Example 1: Basic Generator-as-Coroutine

```
def greeter():
    name = yield "What's your name?"
    yield f"Hello, {name}!"

g = greeter()
print(next(g))        # → "What's your name?"
print(g.send("Alice"))  # → "Hello, Alice!"
```

Here: • yield pauses the function and optionally receives data back via .send(...). • This is a “generator-based coroutine”.

 Coroutine vs Generator

```
Feature                         Generator (def + yield)
Coroutine (async def)
--------------                -------------------------------                    ------
------------------------

1. Keyword                         yield
await, async def

2. Resume with                     next(), .send()
await, event loop

3. Use case                         Data pipelines, lazy iteration
Async I/O, concurrency

4. Syntax                         Synchronous
Asynchronous
```

 Example 2: Async Coroutine (Modern Python)

```
import asyncio

async def download():
    await asyncio.sleep(1)
    return "Download complete"

async def main():
    result = await download()
    print(result)

asyncio.run(main())
```

Here: • async def defines a coroutine function. • await pauses until the awaited task completes (e.g., network delay, file read).

 Use Cases • Network I/O: HTTP requests, WebSocket handling • File I/O: reading large files asynchronously • Cooperative multitasking: doing many things without threads • Pipelines: coroutines that push/pull data

```
class CustomAwaitable:
    def __await__(self):
        print("Inside __await__")
        yield  # pause here
        return "Done!"

async def main():
    result = await CustomAwaitable()
    print(result)

asyncio.run(main())
```

 Output

```
Inside __await__
Done!
```

 Why Does This Work?

 Python allows any object with a **await**() method to be await-ed.

That’s not an exception — it’s part of the official awaitable protocol.

According to PEP 492 (async/await), an object is awaitable if: • It is a coroutine object, OR • It has a **await**() method that returns an iterator

 Let’s Look at This Line:

```
await CustomAwaitable()
```

Here’s what Python internally does: 1. Calls CustomAwaitable().**await**() → this returns a generator. 2. That generator is driven by the event loop (asyncio.run()), as if by repeated next() calls. 3. The yield inside the generator causes it to pause. 4. When the generator is complete, the return value ("Done!") is returned from await.

 What Makes This Legitimate?

Because the **await**() method returns a generator, and generators can be used in an async context if they’re returned by **await**, it is completely legal to use this as a custom awaitable.

This allows developers to define their own awaitable behaviors — useful in async frameworks, mocking, or custom scheduling.

 Confirming the Type

```
obj = CustomAwaitable()
print(hasattr(obj, '__await__'))  # True
print(type(obj.__await__()))      # <class 'generator'>
```

 Using It Manually

You can drive it manually too:

```
g = CustomAwaitable().__await__()
print(next(g))   # None (yielded)
# print(next(g))  # StopIteration: Done!
```

 So Is This an Exception? • Yes, it’s unusual: Most things you await are coroutine functions. • No, it’s not a hack or unintended: Python intentionally supports this via the **await**() protocol.

 TL;DR

Question	Answer
1. Can we await this?	✅ Yes — because it defines __await__() that returns a generator
2. Is this common?	⚠️ Rare in real-world apps — mostly in advanced frameworks
3. Is it an exception?	❌ No — it follows Python's async protocol (__await__)
4. What's the output?	Inside __await__, then Done!
5. Is the generator in __await__() automatically run?	✅ Yes — by the event loop

These concepts — generator, iterator, and iterable — are closely related but not the same

🔍 Short Summary

Term	Is it Iterable?	Is it Iterator?	Can use for loop?	Can use next()?
1. Iterable	✅ Yes	❌ Not necessarily	✅ Yes	❌ No (unless it's also an iterator)
2. Iterator	✅ Yes	✅ Yes	✅ Yes	✅ Yes
3. Generator	✅ Yes	✅ Yes	✅ Yes	✅ Yes

🧠 Definitions

1. Iterable

An object that can be looped over (e.g., for x in obj). 🔁 Must implement **iter**(), which returns an iterator.

📦 Examples: list, tuple, dict, str, generator, custom classes with **iter**.

```
lst = [1, 2, 3]
for i in lst:
    print(i)
```

You can get an iterator from it:

```
it = iter(lst)  # calls lst.__iter__()
```

2. Iterator

An object that produces values one at a time using **next**().

 Must implement both: • **iter**() → returns self • **next**() → returns the next item (raises StopIteration at the end)

```
it = iter([1, 2, 3])  # this is an iterator
print(next(it))  # 1
print(next(it))  # 2
```

3. Generator

A special kind of iterator created using a function with yield.

 Automatically: • Implements **iter**() and **next**() • Maintains internal state between calls

```
def my_gen():
    yield 1
    yield 2
    yield 3

g = my_gen()        # g is a generator (and also an iterator)
print(next(g))      # 1
print(next(g))      # 2
```

 Key Differences

Feature	Iterable	Iterator
1. Used in for	Yes	Yes
2. Has __iter__()	Yes	Yes
3. Has __next__()	Not required	Yes
4. Can be reused	Usually (e.g. list)	No — exhausted after use
5. Needs iter()	Yes	Already ready

 Examples to Compare

A. Iterable but Not Iterator

```
lst = [1, 2, 3]          # list is iterable
it = iter(lst)           # this is now an iterator

print(hasattr(lst, '__next__'))  # False
print(hasattr(it, '__next__'))   # True
```

B. Generator is Iterator & Iterable

```
def gen():
    yield 1
```

```
g = gen()

print(hasattr(g, '__iter__'))  # True
print(hasattr(g, '__next__'))  # True
```

 Summary

Type	Reusable?	Notes
1. List, Tuple	✅ Yes	Can re-iterate many times
2. Generator	❌ No	One-shot, must recreate
3. Iterator	❌ No	Usually one-shot

Explanation of the following Code which uses await with a generator object

!! WARNING! This style of using coroutine as a generator is no longer supported in modern Python

(i.e Python 3.6+), it is deprecated and should not be used at all

Below is just code snippet to demonstrate how it used to work

```
from collections import deque
from types import coroutine

friends = deque(['Ram', 'Sita', 'Laxman', 'Shatrughan', 'Bharat', 'Hanuman'])

@coroutine
def friend_upper():
    while friends:
        friend = friends.popleft().upper()
        greeting = yield
        print(f'{greeting} {friend}')

async def greet(g):
    await g

greeter = greet(friend_upper())
greeter.send(None)
greeter.send('Hello')
greeter.send('Hi')
greeter.send('Hey')
greeter.send('What')
greeter.send('Why')
```

How is a generator (via @coroutine) being awaited in await g?

The short answer is:

Because @coroutine turns a generator into an awaitable coroutine — by giving it a **await**() method under the hood.

 Let’s unpack this step by step

 Code Recap:

```
from collections import deque
from types import coroutine

friends = deque(['Ram', 'Sita', 'Laxman', 'Shatrughan', 'Bharat', 'Hanuman'])

@coroutine
def friend_upper():
    while friends:
        friend = friends.popleft().upper()
        greeting = yield
        print(f'{greeting} {friend}')
```

You’re using:

```
@coroutine
```

from:

```
from types import coroutine
```

from types import coroutine

```
This decorator transforms a generator-based coroutine into something that can be used
with await.
```

 What does @coroutine do?

It marks the generator function so that it behaves like an awaitable object — that is: • It adds a **await**() method to the generator object. • This makes it legal to use with await.

So this line:

```
await g
```

works, because: • g is the generator returned by friend_upper() • @coroutine makes g.**await**() available • Python’s event loop knows how to drive **await**() step-by-step

This is how asyncio used to work before async def/await syntax was introduced (i.e., Python 3.4 and earlier Python 3.5 styles).

 Under the Hood: What’s Happening

```
@coroutine
def friend_upper():
...
```

Becomes:

```
def friend_upper():
...
friend_upper = types.coroutine(friend_upper)
```

Internally, this creates a generator object with a . **await**() method that delegates to the generator itself using yield from.

 What Happens During Execution?

```
greeter = greet(friend_upper())
```

```
•    friend_upper() returns a generator (but marked as awaitable)
•    greet() is a coroutine that awaits this generator
```

```
greeter.send('Hello') # greeting = 'Hello'
```

Causes:

```
print(f'{greeting} {friend}')
```

 No, you cannot use await in a function that is not declared with async def.

Trying to use await in a regular def function will result in a SyntaxError.


```
def regular_function():
    await some_coroutine()  #  Invalid

# SyntaxError: 'await' outside async function
```

Python requires that any use of await must be inside an async def function — this is enforced at the syntax level by the parser.

 Correct Usage

```
async def my_async_function():
    await some_coroutine()
```

And then you run it via:

```
import asyncio
asyncio.run(my_async_function())
```

 Why This Restriction Exists • await only makes sense in the context of a coroutine. • The event loop must manage the paused/resumed state of the function, which requires it to be async def. • Regular def functions return values immediately and can’t suspend execution.

 TL;DR

Question	Answer
1. Can you use await inside a regular def function?	No — it’s a syntax error
2. What kind of function can use await?	Only async def functions
3. Why?	Because only async def functions are coroutines that can pause/resume under control of the event loop

without create_task, can we run multiple coroutines at the same time, meaning if we simply await 2 co routines without creating tasks for them, will they run concurrerntly?

Great question — you’re digging into a key concept in Python’s asyncio: what makes coroutines run concurrently?

 Short Answer

 No, if you await two coroutines sequentially without using create_task() or gather(), they will not run concurrently.

They will run one after the other, just like synchronous code — because await waits for the first to finish before moving to the next.

```
import asyncio

async def task1():
    print("Start task1")
    await asyncio.sleep(1)
    print("End task1")

async def task2():
    print("Start task2")
    await asyncio.sleep(1)
    print("End task2")

async def main():
    await task1()  #  waits until task1 is fully done
    await task2()  #  starts only after task1 is done
```

```
asyncio.run(main())
```

Output:

```
Start task1
End task1
Start task2
End task2
```

 Total time: ~2 seconds → Sequential execution

Concurrent Execution Without create_task: Use asyncio.gather

```
async def main():
    await asyncio.gather(task1(), task2())  #  runs concurrently

# OR, more explicitly:
# t1 = task1()
# t2 = task2()
# await asyncio.gather(t1, t2)
```

Output:

```
Start task1
Start task2
End task1
End task2
```

 Total time: ~1 second → Concurrent execution

 Or Use asyncio.create_task (Explicit task scheduling)

```
async def main():
    t1 = asyncio.create_task(task1())
    t2 = asyncio.create_task(task2())
    await t1
    await t2
```

 TL;DR

```
Pattern                                    Concurrent?          Notes
-----------------------------------------  --------------      ------------------
1. await task1(); await task2()               No               Runs one after
the other

2. await asyncio.gather(task1(), task2())     Yes              Best for
launching coroutines together

3. create_task + await                        Yes              Lower-level,
used for long-lived tasks
```

 Final Note:

```
await = wait for one thing

asyncio.gather / create_task = start multiple things, then await them together
```

Let’s walk through a complete, detailed example of using the unsync library to run:

```
•    Synchronous functions
•    Asynchronous coroutines
•    CPU-bound functions (processes)
•    All in parallel
```

We’ll show: • How to declare each type • What each returns (UnsyncFuture) • How to access the results • What’s happening under the hood

 Overview

```
Type of Task                    Decorator Syntax               Backed by
Returns
-------------------------      -------------------------      --------------------
----------------

Sync I/O-bound (default)        @unsync                        ThreadPoolExecutor
UnsyncFuture
CPU-bound (heavy compute)       @unsync(process=True)          ProcessPoolExecutor
UnsyncFuture
Async coroutine                 @unsync on async def           asyncio event loop
UnsyncFuture
```

 Step 2: Full Example

```
from unsync import unsync
import time
import asyncio
import math

# --- Threaded: I/O-style sleep ---
@unsync
def sleep_task(name):
    time.sleep(1)
    return f"Done sleeping: {name}"

# --- Async: asyncio-style task ---
@unsync
async def async_task(name):
    await asyncio.sleep(1)
    return f"Done async: {name}"

# --- Process: CPU-bound task ---
@unsync(process=True)
```

```
def cpu_task(x):
    # Simulate CPU-intensive operation
    return f"CPU result: sqrt({x}) = {math.sqrt(x)}"

# --- Run all tasks together ---
def main():
    futures = [
        sleep_task("Threaded"),       # Threaded function
        async_task("Async"),          # Async coroutine
        cpu_task(1000000),            # CPU-bound process
    ]

    # These are all UnsyncFuture objects
    for f in futures:
        print(f"Type: {type(f)}")     # Shows <class 'unsync.future.UnsyncFuture'>

    # Wait for all to complete and collect results
    results = [f.result() for f in futures]
    print("\nResults:")
    for r in results:
        print(r)

if __name__ == '__main__':
    main()
```

 Output

```
Type: <class 'unsync.future.UnsyncFuture'>
Type: <class 'unsync.future.UnsyncFuture'>
Type: <class 'unsync.future.UnsyncFuture'>

Results:
Done sleeping: Threaded
Done async: Async
CPU result: sqrt(1000000) = 1000.0
```

 Each task runs concurrently: • The sleep_task runs in a thread. • The async_task runs on the async event loop. • The cpu_task runs in a separate process.

They all return an UnsyncFuture, which supports:

```
Method / Property                                Meaning
-----------------------------                    ---------------------------------
-------
1. .result()                                     Wait and return the result
(blocking)

2. .done()                                       True if completed

3. .exception()                                  Returns exception if raised

4. .add_done_callback(fn)                        Call when future completes
```

 Under the Hood

@unsync • On def: runs in ThreadPoolExecutor • On async def: runs on asyncio event loop • On def with process=True: runs in ProcessPoolExecutor

 Bonus: Non-blocking Access with .add_done_callback

```
f = sleep_task("Ram")
f.add_done_callback(lambda fut: print("Callback:", fut.result()))
```

You can even use fut.result(timeout=...) to control blocking behavior.

 Summary Table of Unsync Usage

```
What You Want                                   Decorator
Returns                 Runs In
----------------------------------------       ----------------------          ------
-----------       ----------------------
1. Async coroutine                              @unsync on async def
UnsyncFuture            asyncio event loop

2. I/O-style blocking (e.g., time.sleep)        @unsync on def
UnsyncFuture            ThreadPoolExecutor

3. Heavy computation                            @unsync(process=True)
UnsyncFuture            ProcessPoolExecutor
```

 TL;DR

• unsync simplifies concurrency without managing threads, event loops, or futures manually. • All functions return UnsyncFuture regardless of backend. • .result() blocks until completion and returns the result. • All tasks run concurrently, giving performance and simplicity.

Can we prime a coroutine by using next()?

 Yes — but only for regular generator-based coroutines, not async def coroutines.

Let’s break it down very clearly, because it depends entirely on the kind of coroutine you’re talking about:

 1. Generator-based coroutine (@coroutine + yield)

Yes, you can prime it using next() — because it’s still a generator under the hood.

```
from types import coroutine

@coroutine
def gen_coro():
    print("Primed")
    yield "Paused here"

g = gen_coro()

print(next(g))  #  works: "Primed" then yields "Paused here"
```

 Why this works: • @coroutine transforms a generator into an awaitable coroutine, but it’s still a generator object. • So next(g) and g.send(...) still work. • await g also works.

 2. Native coroutine (async def)

 No, you cannot use next() on a coroutine defined with async def.

```
async def native_coro():
    print("Inside")
    return 42

c = native_coro()

next(c)  #  TypeError: 'coroutine' object is not an iterator
```

 Why this fails: • Native coroutines (created with async def) are not iterators. • They must be awaited (await c) or wrapped in an asyncio task.

 Summary Table

```
Type of Function                Can you next() it?              Can you await it?
-----------------               ----------------------          ----------------------
----

1. def + yield                  Yes                             No

2. @coroutine + yield           Yes                            Yes

3. async def                    No                              Yes

4. async def + yield            No (not iterable)               No (async for) only
```

 Final Verdict

You can prime a coroutine with next() only if it’s a generator-based coroutine, such as one decorated with @coroutine. You cannot use next() on a coroutine created via async def.

so for most generators, we can use next/send interchangeably ?

Excellent question — and the short answer is:

 Yes, for most generators, you can use .next() and .send(None) interchangeably — but there are important nuances you should know.

Let’s break it down clearly and precisely:

 When next() and send(None) Are Equivalent

 Basic generator:

```
def my_gen():
    yield "step 1"
```

```
    yield "step 2"

g = my_gen()

print(next(g))        #  "step 1"
print(g.send(None))   #  "step 2" — works like next()
```

 send(None) is exactly the same as next() when: • The generator is waiting at a bare yield • You don’t want to inject any actual data

 When next() and send(None) Are Not Interchangeable

 Case 1: First call to a generator with yield = (expression)

```
def my_gen():
    name = yield "What's your name?"
    yield f"Hello, {name}"

g = my_gen()

print(next(g))         #  primes the generator, yields "What's your name?"
print(g.send("Alice")) #  sends "Alice" into the generator
```

 But this will fail:

```
g = my_gen()
g.send("Alice")  #  TypeError: can't send non-None value to a just-started generator
```

You must prime the generator first using next() or send(None).

 Case 2: When you want to inject a value into the generator

Then send(...) is the only option:

```
def echo():
    while True:
        val = yield
        print(f"Received: {val}")

g = echo()
next(g)         #  primes to first `yield`
g.send("Hello") #  sends "Hello" into the generator
```

Calling next(g) here again would just send None, which isn’t useful if you’re expecting real data.

 Summary Table

```
Use Case                              Use next()                 Use send(None)
Use send(value)
```

```
1. Prime a new generator                 ✅ Yes                      ✅ Yes
❌ No

2. Resume at a bare yield                ✅ Yes                      ✅ Yes
❌ Only if value is None

3. Pass value to yield expression        ❌ No                       ❌ No
✅ Yes
```

✅ TL;DR • next() is shorthand for send(None) • You must use next() or send(None) to prime a generator (first step) • Use send(value) only after priming, when the generator is paused at name = yield • ❌ Never use send(value) on a just-started generator — you'll get a TypeError

✅ Summary Chart

Task Type	Use `asyncio`?	Use Threads?	Notes
Network requests	✅ Yes	❌ Not needed unless using blocking libraries	
File I/O	❌ No	✅ Yes	Use `asyncio.to_thread()` or `ThreadPoolExecutor`
CPU-bound tasks	❌ No	❌ Use Process Pool	Use `multiprocessing` or `ProcessPoolExecutor`
Mixed I/O (files + network)	⚠️ with care	⚠️ use threads for files	Use async for network, threads for blocking ops

✅ Final Recommendations

Scenario	Tool
Web scraping, API calls	`asyncio` + `aiohttp`
Read/write CSV, JSON, logs	`asyncio.to_thread()` or `ThreadPoolExecutor`
Heavy file parsing or computation	`ProcessPoolExecutor` (CPU-bound)
Want simplicity and flexibility	Use `unsync` to combine all styles

Generators and Coroutines

Understanding the `TypeError: can't send non-None value to a just-started generator`

This error is a classic "rite of passage" when working with generator-based coroutines.

The Problem: A generator function's code does not run until you "prime" it. You cannot send a value (`.send(value)`) to a generator that hasn't started executing and paused at its first `yield` expression.

The Solution: You must first call `next(g)` or `g.send(None)` to advance the generator to its first `yield` point. Only then can it accept a non-`None` value.

Example

```
def generator():
    print("-> Generator started")
    value = yield
    print(f"-> Generator received: {value}")
    yield

gen = generator()

# This will cause the TypeError:
# gen.send("Hello")

# Correct way: Prime the Generator first
print("Priming the generator...")
next(gen)  # or gen.send(None)
print("Generator is primed and waiting at the first yield.")

# Now we can send a value
print("Sending value to generator...")
gen.send("Hello")
```

Output:

```
Priming the Generator...
-> Generator started
Generator is primed and waiting at the first yield.
Sending value to generator...
-> Generator received: Hello
```

```
Above is a generator not a co-routine
```

What is a Coroutine?

> 🧠 ***In Simple Terms:*** *Think of a coroutine as a "function that can take a break." While normal functions run from start to finish without stopping, coroutines can pause (for ex: using `yield`), let other code run, and then pick up exactly where they left off when a value is sent back to them. Co-routines can also use other strategies to pause for ex: await*

> 🚀 ***Why Use Coroutines?*** *They are incredibly efficient for handling asynchronous tasks, such as:*
>
> - *Making API calls*
> - *Reading from files or network sockets*
> - *Waiting for user input*
>
> *They help you write non-blocking code that can handle many operations concurrently.*

A Classic Coroutine Example

Here is a simple coroutine that yields a value and then waits to receive a value back.

```
async def s_coroutine():
    print('Coroutine started')
    await asyncio.sleep(2)
    print('Coroutine resumes working')

def simple_coroutine():
    print("Coroutine started")
    x = yield 42  # Pauses here, returns 42, and waits for a value to be sent
    print(f"Received: {x}")

# 1. Create the coroutine object
co = simple_coroutine()

# 2. Prime the coroutine
#    - It runs up to the first yield.
#    - It prints "Coroutine started".
#    - It yields the value 42.
value_from_yield = next(co)
print(f"Value from first yield: {value_from_yield}")

# 3. Send a value back into the coroutine
#    - The coroutine resumes.
#    - The value 100 is assigned to 'x'.
#    - It prints "Received: 100".
#    - The function finishes, raising StopIteration.
try:
    co.send(100)
except StopIteration:
    print("Coroutine finished.")
```

Key Coroutine Concepts

Concept	Meaning
`yield`	In a coroutine, it pauses the function, returns a value, and waits for a value to be sent back.
`send(value)`	Resumes the coroutine from its last `yield` point and passes `value` into it.
`next()`	Starts or resumes the coroutine until the next `yield`. It's equivalent to `send(None)`.
`await`	Used in modern Python (3.5+) with `async def` functions to pause execution on other awaitable objects.

1. Can a coroutine accept and return data at the same time? Yes, this is the core feature of a classic generator-based coroutine! However, it's not happening in

the exact same instant. It's a two-step sequence that happens at a single point in your code: the yield expression.

Returning Data: When a coroutine's execution reaches a yield statement, it yields (returns) a value to the caller and pauses its execution at that exact spot. Accepting Data: The caller can then use the .send() method to resume the coroutine. The value passed to .send() becomes the result of the yield expression inside the coroutine, effectively passing data into it. The simple_coroutine example from your python_secrets.md file is perfect for illustrating this:

```
def simple_coroutine():
    print("-> Coroutine started")
    # This is the key line:
    # 1. It yields the value 42 and pauses.
    # 2. When resumed, the value sent in is assigned to 'x'.
    x = yield 42
    print(f"-> Received: {x}")

# --- Execution Flow ---

# 1. Create the coroutine; no code has run yet.
co = simple_coroutine()

# 2. Prime the coroutine by calling next().
#    - It runs until the `yield 42`.
#    - It prints "-> Coroutine started".
#    - It RETURNS the value 42 and pauses.
value_from_yield = next(co)
print(f"Caller received: {value_from_yield}")

# 3. Send data back into the paused coroutine.
#    - The coroutine resumes.
#    - The value 100 is ACCEPTED and assigned to 'x'.
#    - It prints "-> Received: 100".
#    - The function ends, raising StopIteration.
try:
    co.send(100)
except StopIteration:
    print("-> Coroutine finished.")
```

Output:

```
-> Coroutine started
Caller received: 42
-> Received: 100
-> Coroutine finished.
```

So, a single x = yield 42 line both sends data out (42) and, upon resuming, receives data in (100).

2. Is a thread not better suited than a coroutine for handling user input? For handling a standard, blocking input() call, yes, a thread is almost always a better and more straightforward solution than a coroutine.

Here’s why:

The Threading Approach (Simpler and More Direct) The input() function is a blocking operation. It halts the execution of its thread until the user types something and presses Enter.

By placing the input() call in a separate thread, you allow your main program to remain responsive and perform other computations while the dedicated input thread is blocked. This is a classic and highly effective use case for threading.

Your own example code in 1_threads.py demonstrates this perfectly. The complex_calculation can run at the same time the program is waiting for user input.

```
# Based on /Users/ravikumarnayak/personal_projects/python/The-Complete-Python-Course/13_async_development/sample_code/1_threads.py
from threading import Thread
import time

def ask_user():
    user_input = input('Enter your name: ')
    print(f'Hello, {user_input}')

def complex_calculation():
    print('Started calculating...')
    [x**2 for x in range(20000000)]
    print('Finished calculating.')

thread1 = Thread(target=complex_calculation)
thread2 = Thread(target=ask_user)

start = time.time()
thread1.start()
thread2.start()

thread1.join()
thread2.join()
print(f'Two thread total time: {time.time() - start:.2f}s')
```

When you run this, the "Started calculating..." message appears immediately, and you can type your name while the calculation happens in the background. The total time will be that of the longest task, not the sum of both.

The Coroutine (asyncio) Approach (More Complex) Modern coroutines with async/await run on a single-threaded event loop. If you call a blocking function like input() inside an async function, it will block the entire event loop. No other coroutines can run, and your application will freeze, defeating the entire purpose of asyncio.

The Wrong Way (Don't do this!):

```
import asyncio

async def ask_user_badly():
    # This will freeze the event loop!
    user_input = input('Enter your name: ')
```

```
    print(f'Hello, {user_input}')

async def other_task():
    print("Other task starting...")
    await asyncio.sleep(2)
    print("Other task finished.")

# If you run these together, other_task will not start its sleep
# until AFTER the user has provided input.
# await asyncio.gather(ask_user_badly(), other_task())
```

The Correct (but more complex) Way:

To properly handle a blocking call in asyncio, you must run it in a separate thread managed by an executor, which prevents it from blocking the main event loop.

```
import asyncio
import time

def blocking_ask_user():
    """A standard function with a blocking call."""
    user_input = input('Enter your name: ')
    print(f'Hello, {user_input}')

async def main():
    loop = asyncio.get_running_loop()
    start = time.time()

    # Schedule the blocking function to run in the default thread pool executor
    input_task = loop.run_in_executor(
        None, blocking_ask_user
    )

    # Create another concurrent task
    calculation_task = asyncio.sleep(3) # Simulates a 3-second async operation

    print("Calculation and user input are running concurrently.")
    await asyncio.gather(input_task, calculation_task)
    print(f"Total time: {time.time() - start:.2f}s")

asyncio.run(main())
```

As you can see, while it's possible with asyncio, it requires the extra step of using run_in_executor. For the simple case of handling input(), the threading module provides a much more direct and readable solution.

###If I ran a loop for a co routine, would it be able to accept data and return data at the same time? give me an example?

Yes, a coroutine running in a loop can absolutely accept data and return data in a continuous cycle. The key is the yield expression, which acts as a two-way communication channel.

When the caller uses .send(value), it sends data into the coroutine. The yield expression inside the coroutine receives that value, processes it, and then yields a new value back to the caller. This creates a "ping-pong" effect where the caller's loop and the coroutine's loop are in sync, exchanging data on each iteration.

Example: A Running Average Coroutine Here is a practical example. We'll create a coroutine that runs in an infinite loop. On each iteration, it accepts a number, updates its internal state, and returns the new running average.

```
def running_averager():
    """
    A coroutine that maintains a running average.
    It accepts a number and yields the current average.
    """
    print("-> Coroutine started, ready to receive values.")
    # Initialize internal state
    total = 0.0
    count = 0
    average = None

    while True:
        # The magic happens here:
        # 1. The coroutine yields the `average` and PAUSES.
        # 2. When the caller .send()s a value, it RESUMES.
        # 3. The sent value is assigned to `term`.
        term = yield average

        # Process the received data and update state
        print(f"   -> Coroutine received: {term}")
        total += term
        count += 1
        average = total / count

# --- The Caller Code ---

# 1. Create the coroutine object.
averager = running_averager()

# 2. Prime the coroutine. This is essential!
#    We call next() to run the code up to the first `yield`.
#    It will yield its initial `average` (which is None) and then pause.
#    The documentation in your `python_secrets.md` file explains this perfectly.
next(averager)

# 3. Now, loop and send data to the coroutine.
#    The .send() call both sends a value and receives the next yielded value.
for number in [10, 20, 60, 30]:
    print(f"Caller: Sending {number}...")
    # Send the number in and get the calculated average back.
    current_average = averager.send(number)
    print(f"Caller: Current average is {current_average:.2f}\n")

# 4. Close the coroutine to clean up (good practice).
```

```
averager.close()
print("-> Coroutine closed.")
```

Execution Breakdown

Let's trace the execution to see how the data flows:

averager = running_averager(): Creates the coroutine object, but no code inside it runs yet. next(averager): Primes the coroutine. -> Coroutine started... is printed. The while True loop starts. It hits term = yield average. It yields the value of average (which is None) and pauses, waiting for data. averager.send(10): The caller sends 10 into the paused coroutine. 10 is assigned to the term variable. -> Coroutine received: 10 is printed. total becomes 10, count becomes 1, average becomes 10.0. The while loop repeats, hitting yield average again. It yields 10.0 back to the caller and pauses. The caller receives 10.0, assigns it to current_average, and prints it. averager.send(20): The caller sends 20. 20 is assigned to term. -> Coroutine received: 20 is printed. total becomes 30, count becomes 2, average becomes 15.0. The coroutine yields 15.0 back to the caller and pauses. The caller receives 15.0 and prints it. This cycle continues, with the coroutine maintaining its state (total and count) across multiple send calls, which is something a normal function cannot do.

```
-> Coroutine received: 20 is printed.
total becomes 30, count becomes 2, average becomes 15.0.
The coroutine yields 15.0 back to the caller and pauses.
The caller receives 15.0 and prints it. This cycle continues, with the coroutine
maintaining its state (`total` and `count`) across multiple `send` calls, which is
something a normal function cannot do.

---
```

A function can be async and a generator at the same time

Asynchronous Generators (`async def` + `yield`)

Since Python 3.6, a function **can** be both `async` and a generator. This powerful feature, called an **asynchronous generator**, allows you to `await` asynchronous operations and then `yield` a result in a sequence.

This is perfect for streaming data that requires an async call for each chunk (e.g., fetching pages from a web API).

How It Works

- It is defined with `async def`.
- It uses `yield` to produce values.
- It can use `await` to pause for other coroutines.
- It is consumed using an `async for` loop.

Example

Here is an asynchronous generator that counts down, waiting for one second between each number it yields.

```
import asyncio

```

```python
async def async_countdown(n):
    """
    An asynchronous generator that counts down from n,
    waiting 1 second between each yield.
    """
    print("-> Countdown started!")
    while n > 0:
        await asyncio.sleep(1)  # Pauses for an async operation
        yield n                 # Yields a value
        n -= 1
    print("-> Countdown finished!")

async def main():
    # You consume an async generator with `async for`
    print("Starting the async for loop...")
    async for number in async_countdown(3):
        print(f"   Received number: {number}")
    print("Async for loop finished.")

# Run the main coroutine
asyncio.run(main())
```

Output:

```
Starting the async for loop...
-> Countdown started!
   Received number: 3
   Received number: 2
   Received number: 1
-> Countdown finished!
Async for loop finished.
```

is a coroutine always defined using async?

Great question!

 No, a coroutine in Python is not always defined using async def — though that is the modern and preferred way.

There are two types of coroutines in Python:

 1. Modern Coroutines — using async def (Python 3.5+)

These are what most people mean when they say “coroutine” today.

```python
async def say_hi():
    return "Hello"

coro = say_hi()
print(type(coro))  # <class 'coroutine'>
```

You can await these:

```
import asyncio
asyncio.run(say_hi())
```

First, What Is an Async Generator?

```
async def my_async_gen():
    yield 1
    yield 2
```

This defines an async generator function, which returns an async generator object.

You consume it using async for, not await.

Can You await an Async Generator?

No, you cannot directly await an async generator.

If you try:

```
await my_async_gen()  # TypeError
```

You'll get:

```
TypeError: 'async_generator' object is not awaitable
```

You must use:

```
async for val in my_async_gen():
    print(val)
```

Answer to Your Questions

1. Can we yield a value back to the caller while it is awaiting the generator?

Yes — but the caller must be using async for to receive values.

Example:

```
async def counter():
    for i in range(3):
        print(f"Yielding {i}")
        yield i

async def main():
    async for val in counter():
        print(f"Received: {val}")
```

So: • You're not "awaiting the generator" per se. • The caller awaits each iteration of the generator with async for.

2. Can the caller send a value to the async generator using .send()?

 Not really — this is not supported in Python's async generators as of Python 3.13.

While regular (sync) generators support .send() to inject values, async generators do not.

If you try:

```
agen = counter()
await agen.asend("hello")
```

It raises:

```
TypeError: can't send non-None value to a just-started async generator
```

Even after it starts, using .asend(val) is clunky and only works in very narrow cases. It's rarely used and has limited support — Python explicitly discourages this.

So:

 Use yield to emit values Do not expect to .send() values into an async generator

 Why No send() for Async Generators?

Because it would require the event loop to handle bidirectional streaming in a very precise way. This is complex and Python opted to keep async generators simpler — one-way flow only.

```
This is a design choice — async generators are meant for pull-style streams (e.g.
async for), not push-pull interaction.
```

 TL;DR

```
Question                                                  Answer
--------------------------------------------------        ----------------------
----------------------------
1. Can you await an async generator?                      No — use async for
instead

2. Can it yield values to the caller?                     Yes — via yield in
async def, consumed by async for

3. Can the caller send values back (.send())?             No — async
generators do not support .send()
```

 2. Generator-based Coroutines — using @types.coroutine (Legacy Style)

Before async def, coroutines were built with yield + @coroutine.

```
The @coroutine decorator (from types or asyncio) predates async def and was introduced
in Python 3.4, before async/await syntax existed.
```

```
from types import coroutine

@coroutine
def legacy_coro():
    yield
    return 42

c = legacy_coro()
print(type(c))  # <class 'generator'>
```

These behave like coroutines, but are actually generators with a special **await**() method under the hood.

They can be awaited too:

```
async def main():
    result = await legacy_coro()
    print(result)

import asyncio
asyncio.run(main())
```

 Important Notes • @coroutine is deprecated since Python 3.8. • Removed from asyncio.coroutine in Python 3.11. • You should avoid it in new code and use async def with await or async for.

 Want Bidirectional Async Streams?

Use asyncio.Queue to simulate a send/receive async channel — safe, modern, and works perfectly with async def.

 Summary Table

```
Definition Style                    Awaitable?              Common Today?
Underlying Type
-----------------                   -----------------       -------------------------
--    ----------------------------

1. async def                        Yes                 Yes
coroutine

2. def + yield + @coroutine         Yes                 Rare
generator with __await__()

3. def + yield (no decorator)       No                  Yes (but not a coroutine)
generator
```

 TL;DR

```
•    A coroutine is not always defined with async def
•    But in modern Python, you should always use async def unless you’re writing low-level async libraries
•    @coroutine is still useful for creating awaitable wrappers around generators, but it’s mostly legacy
```

What Is an Event Loop?

The event loop is a mechanism that waits for tasks (coroutines, I/O events, timers, etc.) to be ready, and then runs them one at a time in a loop — cooperatively, not in parallel.

Think of it like a scheduler that: • Keeps track of what needs to run • Runs whatever is ready without blocking • Suspends tasks that are waiting (e.g., sleeping, I/O) • Resumes tasks when their data is ready

Analogy

Imagine a single-threaded robot that: • Checks all its tasks in a list • Runs each task until it hits an “await” • Puts it aside and goes to the next ready task • Comes back later when the paused task can continue

1. Do tasks run concurrently via cooperative multitasking?

Yes.

asyncio uses cooperative multitasking: • Each async def coroutine must explicitly yield control by hitting an await. • When that happens, the coroutine pauses itself, and the event loop decides what to do next: • Resume another coroutine that is ready. • Wait for an I/O event. • Handle timeouts, etc.

So the task gives up control voluntarily by awaiting something — the event loop does not preempt it.

2. Can the event loop suspend a task that is synchronous or blocking?

No.

The event loop cannot preempt or forcibly suspend a blocking synchronous function.

That’s a key limitation of asyncio: • If a coroutine calls a blocking function (like time.sleep() or input()), it blocks the entire event loop. • The event loop can’t switch away from it — because it only works with tasks that cooperate.

Even though there’s only one event loop thread, multiple async tasks can run concurrently because they cooperate by yielding control when they are waiting (e.g., on I/O, sleep, or any awaitable).

They don’t run in parallel on CPU, but they take turns running — that’s concurrency (not parallelism).

 Important Distinction

```
Term                                                Meaning in Python asyncio
-------------------                                 ---------------------------------
---------------------
1. Concurrency                                      Tasks switch off cooperatively
(e.g., I/O waiting).
2. Parallelism                                      True simultaneous execution on
multiple cores/threads.
                                                    Not provided by default in
asyncio. Use concurrent.futures
                                                    or multiprocessing for that.


concurrency is not parallelism.
```

 Definition of Concurrency

Concurrency is when multiple tasks are in progress at the same time, but not necessarily running at the exact same moment.

Instead of running simultaneously, the tasks take turns — switching between each other so fast that it seems like they’re happening at once.

In Python asyncio, this happens via cooperative multitasking: each task voluntarily pauses (await) so others can run.

 Definition of Parallelism

Parallelism is when multiple tasks are actually executing at the same time, on multiple cores or threads.

This is real simultaneous execution — typical in multi-threading, multi-processing, or using multiple CPUs.

```
 Concurrency vs.  Parallelism


Feature                     Concurrency
Parallelism
------------------          ----------------------------------------       ----------
--------
1. Execution                Tasks interleave, not overlap                  Tasks run
at the same time

2. Threads/Cores            Can be single-threaded (e.g. asyncio)          Requires
multiple threads/cores

3, Use Case                 I/O-bound tasks                                CPU-bound
tasks
```

```
4. Example                  Async web scraping                          Video
rendering on multiple cores
```

 Analogy:

Think of the event loop like a single waiter in a restaurant with many customers (tasks): • Each customer (async task) gives an order and waits. • While waiting for food (I/O), they say: “Come back later.” • The waiter serves the next customer. • Once the food is ready (e.g., file/HTTP/database is done), the waiter returns to the customer.

So even though there’s only one waiter (event loop), multiple customers are handled concurrently — no one just blocks the waiter.

 Basic Event Loop Flow 1. You define one or more coroutines (async def) 2. You submit them to the event loop 3. The loop starts running and manages when each coroutine is allowed to run next 4. When a coroutine awaits, it’s paused 5. The event loop resumes it later when it’s ready

 Example of Event Loop in Action

```
import asyncio

async def say_hello():
    await asyncio.sleep(1)
    print("Hello")

async def say_world():
    await asyncio.sleep(1)
    print("World")

async def main():
    await asyncio.gather(say_hello(), say_world())

asyncio.run(main())  # ← This starts the event loop
```

What Happens: • asyncio.run(main()) → starts the event loop • main() awaits two coroutines concurrently • Event loop sleeps for 1 second, then prints both

 How to Think About It

```
You Call…                                   What It Does
------------------------                    ----------------------------------
-----------------
asyncio.run(...)                            Starts and manages the event loop
automatically

await coro()                                Pauses until the result is ready

asyncio.gather(...)                         Runs multiple coroutines
concurrently
```

```
await asyncio.sleep(n)                                  Suspends for n seconds without
blocking
```

 Behind the Scenes

Python uses selectors under the hood (e.g., epoll, kqueue) to: • Monitor file/network/socket events • Schedule future tasks (via call_later, sleep) • Resume paused coroutines efficiently

The event loop is single-threaded and non-blocking — meaning it can handle thousands of tasks as long as none of them blocks.

 What Blocks the Event Loop?

Any sync or blocking call — like: • time.sleep() • open().read() (for big files) • Long CPU work

Use: • await asyncio.sleep() • asyncio.to_thread(...) • Or delegate CPU-heavy work to ProcessPoolExecutor

 TL;DR — Event Loop Summary

Concept	Summary
What it is	A task scheduler that runs and suspends coroutines
What it runs	Coroutines, tasks, futures, callbacks
When it runs	On asyncio.run(), or when manually started
How it switches	When a coroutine hits await, the loop picks the next ready task
Why it’s useful	Efficient concurrency without threads or blocking

The Role of Each Synchronization Primitive

Think of these tools as different ways for coroutines to coordinate, not all of which involve passing data.

1. asyncio.Event (The Traffic Light) Purpose: Simple signaling. It's a binary flag that one coroutine can set (event.set()) to unblock one or more other coroutines that are waiting for it (await event.wait()). Data Passing: It does not pass data. When a coroutine is unblocked by an event, it simply resumes execution. It doesn't receive any value. It's like a traffic light turning green—it tells you when to go, but doesn't hand you a package.
2. asyncio.Lock (The Key to a Room) Purpose: Mutual exclusion. It ensures that only one coroutine can enter a "critical section" of code at a time. Data Passing: It does not pass data directly. Instead, it protects a shared resource (like a list, dictionary, or object) that holds the data. The coroutines

communicate by modifying this shared resource, and the lock just prevents them from doing it at the same time and causing corruption. The data is on the "whiteboard inside the room," not passed by the key itself.

3. asyncio.Future (The IOU) Purpose: Represents the eventual result of a single, one-time operation. Data Passing: It's a one-way, one-time data transfer. A producer coroutine can set a result on the future (future.set_result(value)), and a consumer can await it to get that single value. Once the result is set, the future is "done" and cannot be used to pass more data. It's like an IOU—you hand it over, and eventually, you get a single payment back. You can't use the same IOU for another payment. Why Queue and .send() Are Different asyncio.Queue (The Conveyor Belt): This is explicitly designed for streaming data. It's a channel where producers can continuously put items and consumers can continuously get them. It's the standard tool for decoupled, many-to-many communication in modern asyncio. Generator .send() (The Phone Call): This is the classic method for tightly coupled, two-way communication with a single generator-based coroutine. yield sends data out, and .send() sends data in. It is inherently "back and forth."

Summary Table

Primitive Purpose Data Flow Analogy asyncio.Queue Streaming data between tasks Many-to-many, continuous Conveyor Belt asyncio.Event Simple signaling None (just a signal) Traffic Light asyncio.Lock Protecting shared state Indirect (via the shared state) Key to a Room asyncio.Future A single eventual result One-way, one-time IOU / Promise Generator .send() Two-way communication One-to-one, back and forth Phone Call

To send data back and forth between multiple workers using async/await

1. asyncio.Queue
2. send(): generators can be used if we use @coroutine type for the async method

Returning data from a pipeline

That's an excellent and very practical question. You've correctly identified that you can't simply return a value from a long-running asynchronous function or pipeline without stopping it.

Practically this means from an async function running a while True loop, we cannot simply return a value, if we do so, it will exit the async task and data pipeline will be broken meaning the producer/consumer will stop

To get data out of a running system, you need to establish a communication channel back to the part of your program that needs the data.

create_task

Yes, asyncio.create_task() schedules the coroutine to start running on the event loop as soon as possible. It doesn't wait for you to await the task object.

Think of it this way:

Creating a Coroutine: my_coro = fetch_data() simply creates a coroutine object. It's like writing a recipe but not starting to cook. Creating a Task: task = asyncio.create_task(my_coro) is like handing that recipe to a chef (the event loop) and saying, "Start working on this as soon as you have a free moment." The chef will begin immediately, and if the recipe says "wait for 10 minutes," the chef won't just

stand there. They will put the first dish in the oven and immediately start working on another recipe you've given them.

Awaiting the task (await task) is you waiting for the chef to tell you that specific dish is ready to serve. The cooking has been happening all along.

asyncio vs unsync - Thread Safety

1. When using pure asyncio, coroutines run within a single-threaded event loop, so it's safe to use asyncio.Queue, which is not thread-safe.
2. However, when using the unsync library, even @unsync async coroutines are executed in background threads (with their own event loops). This means multiple threads may access shared data like a queue.
3. Therefore, we use queue.Queue (which is thread-safe) instead of asyncio.Queue to safely share data between unsync-decorated coroutines and functions.

unsync future object

generate_process_data().result() call.

□

□ What’s Happening in generate_process_data().result()?

1. @unsync Decorator

You’ve defined:

```
@unsync
async def generate_process_data():
    ...
```

This means: • generate_process_data() returns an UnsyncFuture (not a coroutine). • It wraps the async function and runs it in a background thread with its own event loop.

□

2. .result() on UnsyncFuture

UnsyncFuture is a custom object returned by the @unsync decorator.

```
future = generate_process_data()
future.result()
```

This: • Blocks the main thread until the generate_process_data coroutine finishes. • Then returns the final result (if any). • Similar to calling .result() on a concurrent.futures.Future.

□

□ Why Not Just await generate_process_data()?

Because: • generate_process_data() is not a coroutine, it’s an UnsyncFuture object. • You can’t await it unless you’re inside another async function and the coroutine was not decorated with @unsync.

 Summary

```
Code                          What It Does
---------------------------   ------------------------------------------------------
-----------
1. @unsync async def f()         Wraps the async function to run in a background
thread

2. f()                           Returns an UnsyncFuture

3. f().result()                  Blocks until the async function completes and gets
the result

4. await f()                      Invalid – f is not an awaitable coroutine anymore
```

 Final Example in Plain Terms

```
@unsync
async def foo():
    await asyncio.sleep(1)
    return 42

future = foo()          # This runs foo in background thread
result = future.result()  # Wait for foo to finish and get the result (42)
```

Python Async Await Generators Coroutine

```
Concept                            await?               next()?
Notes
-------------                     -------------           -------------
------------

1. def + yield                        
Regular generators

2. async def + await                
Regular coroutines

3. async def + yield                  via async for
Async generators

4. __await__() object                  if it’s a generator
Custom awaitables
```

 1. def + yield

A regular generator (synchronous). You can next() it, but you can’t await it.

```
def gen():
    yield "hello"
    yield "world"
```

```
g = gen()
print(next(g))  # ✅ "hello"
print(next(g))  # ✅ "world"

# await g  ❌ TypeError: cannot 'await' a generator object
```

✅ 2. async def + await

A coroutine function. You must use await, but you can’t next() it.

```
import asyncio

async def say_hello():
    return "Hello Async"

async def main():
    result = await say_hello()  # ✅ works
    print(result)

asyncio.run(main())

# say_hello() is a coroutine
# next(say_hello())  ❌ TypeError
```

✅ 3. async def + yield

This defines an async generator. You can’t await it, but you can iterate with async for.

```
import asyncio

async def async_gen():
    for i in range(3):
        yield i  # ✅ allowed in async generators

async def main():
    async for val in async_gen():  # ✅ works
        print(val)

asyncio.run(main())

# await async_gen() ❌ TypeError: can't be awaited
# next(async_gen()) ❌ TypeError
```

✅ 4. Object with **await**()

Custom object that is awaitable. This can be await-ed and sometimes also next()-ed.

```
class CustomAwaitable:
    def __await__(self):
        print("Inside __await__")
        yield  # pause here
        return "Done!"
```

```
async def main():
    result = await CustomAwaitable()  #  Yes, works
    print(result)

asyncio.run(main())

# Technically, you could also do:
g = CustomAwaitable().__await__()
print(next(g))  #  works because __await__ returns a generator
```

A coroutine is a special kind of function in Python that can: • pause its execution at a certain point, • wait for input or other events, • and then resume from where it left off.

They are used to write non-blocking, asynchronous, and cooperative multitasking code in a clean and readable way.

 Simple Definition:

A coroutine is like a function that can pause, resume, and receive values, making it perfect for concurrent programming without threads.

 Example 1: Basic Generator-as-Coroutine

```
def greeter():
    name = yield "What's your name?"
    yield f"Hello, {name}!"

g = greeter()
print(next(g))        # → "What's your name?"
print(g.send("Alice"))  # → "Hello, Alice!"
```

Here: • yield pauses the function and optionally receives data back via .send(...). • This is a “generator-based coroutine”.

 Coroutine vs Generator

```
Feature                         Generator (def + yield)
Coroutine (async def)
--------------                --------------------------------                    ------
------------------------

1. Keyword                         yield
await, async def

2. Resume with                     next(), .send()
await, event loop

3. Use case                         Data pipelines, lazy iteration
Async I/O, concurrency
```

```
4. Syntax                          Synchronous
Asynchronous
```

 Example 2: Async Coroutine (Modern Python)

```
import asyncio

async def download():
    await asyncio.sleep(1)
    return "Download complete"

async def main():
    result = await download()
    print(result)

asyncio.run(main())
```

Here: • async def defines a coroutine function. • await pauses until the awaited task completes (e.g., network delay, file read).

 Use Cases • Network I/O: HTTP requests, WebSocket handling • File I/O: reading large files asynchronously • Cooperative multitasking: doing many things without threads • Pipelines: coroutines that push/pull data

```
class CustomAwaitable:
    def __await__(self):
        print("Inside __await__")
        yield  # pause here
        return "Done!"

async def main():
    result = await CustomAwaitable()
    print(result)

asyncio.run(main())
```

 Output

```
Inside __await__
Done!
```

 Why Does This Work?

 Python allows any object with a **await**() method to be await-ed.

That’s not an exception — it’s part of the official awaitable protocol.

According to PEP 492 (async/await), an object is awaitable if: • It is a coroutine object, OR • It has a **await**() method that returns an iterator

 Let’s Look at This Line:

```
await CustomAwaitable()
```

Here’s what Python internally does: 1. Calls CustomAwaitable().**await**() → this returns a generator. 2. That generator is driven by the event loop (asyncio.run()), as if by repeated next() calls. 3. The yield inside the generator causes it to pause. 4. When the generator is complete, the return value ("Done!") is returned from await.

 What Makes This Legitimate?

Because the **await**() method returns a generator, and generators can be used in an async context if they’re returned by **await**, it is completely legal to use this as a custom awaitable.

This allows developers to define their own awaitable behaviors — useful in async frameworks, mocking, or custom scheduling.

 Confirming the Type

```
obj = CustomAwaitable()
print(hasattr(obj, '__await__'))  # True
print(type(obj.__await__()))      # <class 'generator'>
```

 Using It Manually

You can drive it manually too:

```
g = CustomAwaitable().__await__()
print(next(g))   # None (yielded)
# print(next(g))  # StopIteration: Done!
```

 So Is This an Exception? • Yes, it’s unusual: Most things you await are coroutine functions. • No, it’s not a hack or unintended: Python intentionally supports this via the **await**() protocol.

 TL;DR

Question	Answer
1. Can we await this?	Yes — because it defines __await__() that returns a generator
2. Is this common?	Rare in real-world apps — mostly in advanced frameworks
3. Is it an exception?	No — it follows Python’s async protocol (__await__)
4. What’s the output?	Inside __await__, then Done!
5. Is the generator in __await__() automatically run?	Yes — by the event loop

These concepts — generator, iterator, and iterable — are closely related but not the same

 Short Summary

```
Term            Is it Iterable?         Is it Iterator?          Can use for loop?
Can use next()?

1. Iterable        Yes                 Not necessarily         Yes
 No (unless it’s also an iterator)

2. Iterator        Yes                 Yes                     Yes
 Yes

3. Generator     Yes                Yes                    Yes                 
Yes
```

 Definitions

1. Iterable

An object that can be looped over (e.g., for x in obj). Must implement **iter**(), which returns an iterator.

 Examples: list, tuple, dict, str, generator, custom classes with **iter**.

```
lst = [1, 2, 3]
for i in lst:
    print(i)
```

You can get an iterator from it:

```
it = iter(lst)  # calls lst.__iter__()
```

2. Iterator

An object that produces values one at a time using **next**().

 Must implement both: • **iter**() → returns self • **next**() → returns the next item (raises StopIteration at the end)

```
it = iter([1, 2, 3])  # this is an iterator
print(next(it))  # 1
print(next(it))  # 2
```

3. Generator

A special kind of iterator created using a function with yield.

 Automatically: • Implements **iter**() and **next**() • Maintains internal state between calls

```
def my_gen():
    yield 1
    yield 2
```

```
    yield 3

g = my_gen()        # g is a generator (and also an iterator)
print(next(g))      # 1
print(next(g))      # 2
```

 Key Differences

```
Feature                         Iterable                        Iterator
------------------             ------------------               ------------------

1. Used in for                  Yes                             Yes

2. Has __iter__()               Yes                            Yes

3. Has __next__()               Not required                    Yes

4. Can be reused               Usually (e.g. list)             No — exhausted after
use

5. Needs iter()                 Yes                             Already ready
```

 Examples to Compare

A. Iterable but Not Iterator

```
lst = [1, 2, 3]          # list is iterable
it = iter(lst)           # this is now an iterator

print(hasattr(lst, '__next__'))  # False
print(hasattr(it, '__next__'))   # True
```

B. Generator is Iterator & Iterable

```
def gen():
    yield 1

g = gen()

print(hasattr(g, '__iter__'))  # True
print(hasattr(g, '__next__'))  # True
```

 Summary

```
Type                        Reusable?                       Notes
----------------           ----------------                ----------------

1. List, Tuple                Yes                           Can re-iterate many
times

2. Generator                  No                           One-shot, must recreate
```

```
3. Iterator                    ❌ No                          Usually one-shot
```

Explanation of the following Code which uses await with a generator object

```
from collections import deque
from types import coroutine

friends = deque(['Ram', 'Sita', 'Laxman', 'Shatrughan', 'Bharat', 'Hanuman'])

@coroutine
def friend_upper():
    while friends:
        friend = friends.popleft().upper()
        greeting = yield
        print(f'{greeting} {friend}')

async def greet(g):
    await g

greeter = greet(friend_upper())
greeter.send(None)
greeter.send('Hello')
greeter.send('Hi')
greeter.send('Hey')
greeter.send('What')
greeter.send('Why')
```

How is a generator (via @coroutine) being awaited in await g?

The short answer is:

Because @coroutine turns a generator into an awaitable coroutine — by giving it a **await**() method under the hood.

✅

🔍 Let’s unpack this step by step

🧠 Code Recap:

```
from collections import deque
from types import coroutine

friends = deque(['Ram', 'Sita', 'Laxman', 'Shatrughan', 'Bharat', 'Hanuman'])

@coroutine
def friend_upper():
    while friends:
        friend = friends.popleft().upper()
        greeting = yield
        print(f'{greeting} {friend}')
```

You’re using:

```
@coroutine
```

from:

```
from types import coroutine
```

from types import coroutine

```
This decorator transforms a generator-based coroutine into something that can be used
with await.
```

🔍

🧠 What does @coroutine do?

It marks the generator function so that it behaves like an awaitable object — that is: • It adds a **await**() method to the generator object. • This makes it legal to use with await.

So this line:

```
await g
```

works, because: • g is the generator returned by friend_upper() • @coroutine makes g.**await**() available • Python’s event loop knows how to drive **await**() step-by-step

This is how asyncio used to work before async def/await syntax was introduced (i.e., Python 3.4 and earlier Python 3.5 styles).

🔍

🔍 Under the Hood: What’s Happening

```

@coroutine
def friend_upper():
...
```

Becomes:

```
def friend_upper():
...
friend_upper = types.coroutine(friend_upper)
```

Internally, this creates a generator object with a . **await**() method that delegates to the generator itself using yield from.

🔍

🔍 What Happens During Execution?

```

greeter = greet(friend_upper())
```

```
•    friend_upper() returns a generator (but marked as awaitable)
•    greet() is a coroutine that awaits this generator
```

```
greeter.send('Hello') # greeting = 'Hello'
```

Causes:

```
print(f'{greeting} {friend}')
```

 No, you cannot use await in a function that is not declared with async def.

Trying to use await in a regular def function will result in a SyntaxError.

```
def regular_function():
    await some_coroutine()  #  Invalid

# SyntaxError: 'await' outside async function
```

Python requires that any use of await must be inside an async def function — this is enforced at the syntax level by the parser.

 Correct Usage

```
async def my_async_function():
    await some_coroutine()
```

And then you run it via:

```
import asyncio
asyncio.run(my_async_function())
```

 Why This Restriction Exists • await only makes sense in the context of a coroutine. • The event loop must manage the paused/resumed state of the function, which requires it to be async def. • Regular def functions return values immediately and can’t suspend execution.

 TL;DR

Question	Answer
1. Can you use await inside a regular def function?	No — it’s a syntax error
2. What kind of function can use await?	Only async def functions
3. Why?	Because only async def functions are coroutines

```
                                                                that can pause/resume
under control of the event loop
```

without create_task, can we run multiple coroutines at the same time, meaning if we simply await 2 co routines without creating tasks for them, will they run concurrerntly?

Great question — you're digging into a key concept in Python's asyncio: what makes coroutines run concurrently?

✅

✅ Short Answer

❌ No, if you await two coroutines sequentially without using create_task() or gather(), they will not run concurrently.

They will run one after the other, just like synchronous code — because await waits for the first to finish before moving to the next.

```
import asyncio

async def task1():
    print("Start task1")
    await asyncio.sleep(1)
    print("End task1")

async def task2():
    print("Start task2")
    await asyncio.sleep(1)
    print("End task2")

async def main():
    await task1()  # ⏳ waits until task1 is fully done
    await task2()  # ⏳ starts only after task1 is done

asyncio.run(main())
```

Output:

```
Start task1
End task1
Start task2
End task2
```

⏱ Total time: ~2 seconds → Sequential execution

Concurrent Execution Without create_task: Use asyncio.gather

```
async def main():
    await asyncio.gather(task1(), task2())  # ✅ runs concurrently

# OR, more explicitly:
# t1 = task1()
```

```
# t2 = task2()
# await asyncio.gather(t1, t2)
```

Output:

```
Start task1
Start task2
End task1
End task2
```

 Total time: ~1 second → Concurrent execution

 Or Use asyncio.create_task (Explicit task scheduling)

```
async def main():
    t1 = asyncio.create_task(task1())
    t2 = asyncio.create_task(task2())
    await t1
    await t2
```

 TL;DR

```
Pattern                                    Concurrent?          Notes
-----------------------------------        --------------      ------------------
1. await task1(); await task2()               No               Runs one after
the other

2. await asyncio.gather(task1(), task2())     Yes              Best for
launching coroutines together

3. create_task + await                        Yes              Lower-level,
used for long-lived tasks
```

 Final Note:

```
await = wait for one thing

asyncio.gather / create_task = start multiple things, then await them together
```

Let’s walk through a complete, detailed example of using the unsync library to run:

```
•    Synchronous functions
•    Asynchronous coroutines
•    CPU-bound functions (processes)
•    All in parallel
```

We’ll show: • How to declare each type • What each returns (UnsyncFuture) • How to access the results • What’s happening under the hood

 Overview

```
Type of Task                    Decorator Syntax               Backed by
Returns
-------------------------      -------------------------      --------------------
---------------

Sync I/O-bound (default)        @unsync                        ThreadPoolExecutor
UnsyncFuture
CPU-bound (heavy compute)       @unsync(process=True)          ProcessPoolExecutor
UnsyncFuture
Async coroutine                 @unsync on async def           asyncio event loop
UnsyncFuture
```

 Step 2: Full Example

```
from unsync import unsync
import time
import asyncio
import math

# --- Threaded: I/O-style sleep ---
@unsync
def sleep_task(name):
    time.sleep(1)
    return f"Done sleeping: {name}"

# --- Async: asyncio-style task ---
@unsync
async def async_task(name):
    await asyncio.sleep(1)
    return f"Done async: {name}"

# --- Process: CPU-bound task ---
@unsync(process=True)
def cpu_task(x):
    # Simulate CPU-intensive operation
    return f"CPU result: sqrt({x}) = {math.sqrt(x)}"

# --- Run all tasks together ---
def main():
    futures = [
        sleep_task("Threaded"),       # Threaded function
        async_task("Async"),          # Async coroutine
        cpu_task(1000000),            # CPU-bound process
    ]

    # These are all UnsyncFuture objects
    for f in futures:
        print(f"Type: {type(f)}")     # Shows <class 'unsync.future.UnsyncFuture'>

    # Wait for all to complete and collect results
    results = [f.result() for f in futures]
    print("\nResults:")
```

```
    for r in results:
        print(r)

if __name__ == '__main__':
    main()
```

 Output

```
Type: <class 'unsync.future.UnsyncFuture'>
Type: <class 'unsync.future.UnsyncFuture'>
Type: <class 'unsync.future.UnsyncFuture'>

Results:
Done sleeping: Threaded
Done async: Async
CPU result: sqrt(1000000) = 1000.0
```

 Each task runs concurrently: • The sleep_task runs in a thread. • The async_task runs on the async event loop. • The cpu_task runs in a separate process.

They all return an UnsyncFuture, which supports:

```
1. exception() method is not directly available on Unsync Future object, it must be
accessed through the future object associated
with the Unsync Future object

2. add_done_callback(callback_fn) is also not directly available on Unsync Future
object, it must be accessed through the future
object associated with the Unsync Future object

3. callback_fn takes Unsync Future object as an argument, and must be provided for the
callback function to be correct

4. Unfuture objects returned by @unsync should not be awaited directly unless you’re
100% sure they’re on the same loop (which is rarely true).

5. await f => where f is of type Unsync Future will result in an error, we must not
await f, we must wait through - f.result()
```

Code Sample below demonstrates how to use unsync library and these methods correctly:

```
import asyncio
from unsync import unsync
import time
import math

@unsync
def sync_task(name):
  print(f'Task started :: {name}')
  print('Sleeping for 2 seconds - synchronous blocking - shall be executed using
ThreadPoolExecutor')
  time.sleep(2)
  print('Exiting the synchronous method')
```

```
  return f'Syncronous Task :: {name}'

@unsync
async def async_task(name):
  print(f'Task started :: {name}')
  print('Asynchronous sleep for 2 seconds - asyncronous non blocking - shall be
executed in an async event loop')
  await asyncio.sleep(2)
  print('Exiting the async method')
  return f'Asynchronous Task :: {name}'

@unsync(process = True)
def cpu_task(name, x):
  print(f'Task started :: {name}')
  print('Task performing cpu bound activity')
  print(f'Sqrt(x) :: {math.sqrt(x)}')
  print('Task exiting')
  return f'CPU Task :: {name}'

def callback_fn(fut):
  print(f'This is a demo callback which will be called when the future completes with
the result :: {fut.result()}')

async def main():
  futures = [
        sync_task('Thread'),
        async_task('Asyncio-Event-Loop'),
        cpu_task('Cpu-Bound-Process', 200000)
    ]
  for f in futures:
    print(f'Callback shall be called when future completes ::
{f.future.add_done_callback(callback_fn)}')
    print(f'Type of future object :: {type(f)}')

  for f in futures:
    result = f.result()
    print(f'Result of Task :: {result}')
    print(f'Task done :: {f.done()}')
    print(f'Exception :: {f.future.exception()}')

asyncio.run(main())
```

```
Method / Property                              Meaning
-----------------------------                  ---------------------------------
-------
1. .result()                                   Wait and return the result
(blocking)

2. .done()                                     True if completed

3. .future.exception()                         Returns exception if raised
```

```
4. .future.add_done_callback(fn)                    Call when future completes
```

 Under the Hood

@unsync • On def: runs in ThreadPoolExecutor • On async def: runs on asyncio event loop • On def with process=True: runs in ProcessPoolExecutor

 Bonus: Non-blocking Access with .add_done_callback

```
f = sleep_task("Ram")
f.add_done_callback(lambda fut: print("Callback:", fut.result()))
```

You can even use fut.result(timeout=...) to control blocking behavior.

 Summary Table of Unsync Usage

```
What You Want                                   Decorator
Returns                 Runs In
----------------------------------------       ----------------------          ------
-----------       ----------------------
1. Async coroutine                              @unsync on async def
UnsyncFuture            asyncio event loop

2. I/O-style blocking (e.g., time.sleep)        @unsync on def
UnsyncFuture            ThreadPoolExecutor

3. Heavy computation                            @unsync(process=True)
UnsyncFuture            ProcessPoolExecutor
```

 TL;DR

• unsync simplifies concurrency without managing threads, event loops, or futures manually. • All functions return UnsyncFuture regardless of backend. • .result() blocks until completion and returns the result. • All tasks run concurrently, giving performance and simplicity.

Can we prime a coroutine by using next()?

 Yes — but only for regular generator-based coroutines, not async def coroutines.

Let’s break it down very clearly, because it depends entirely on the kind of coroutine you’re talking about:

 1. Generator-based coroutine (@coroutine + yield)

Yes, you can prime it using next() — because it’s still a generator under the hood.

```
from types import coroutine

@coroutine
def gen_coro():
    print("Primed")
    yield "Paused here"
```

```
g = gen_coro()

print(next(g))  # ✅ works: "Primed" then yields "Paused here"
```

✅ Why this works: • @coroutine transforms a generator into an awaitable coroutine, but it’s still a generator object. • So next(g) and g.send(...) still work. • await g also works.

🔹 2. Native coroutine (async def)

❌ No, you cannot use next() on a coroutine defined with async def.

```
async def native_coro():
    print("Inside")
    return 42

c = native_coro()

next(c)  # ❌ TypeError: 'coroutine' object is not an iterator
```

❌ Why this fails: • Native coroutines (created with async def) are not iterators. • They must be awaited (await c) or wrapped in an asyncio task.

📌 Summary Table

```
Type of Function                Can you next() it?              Can you await it?
-----------------               ----------------------          ----------------------
----

1. def + yield                  ✅ Yes                          ❌ No

2. @coroutine + yield           ✅ Yes                          ✅ Yes

3. async def                    ❌ No                           ✅ Yes

4. async def + yield            ❌ No (not iterable)            ✅ No (async for) only
```

🧠 Final Verdict

You can prime a coroutine with next() only if it’s a generator-based coroutine, such as one decorated with @coroutine. You cannot use next() on a coroutine created via async def.

so for most generators, we can use next/send interchangeably ?

Excellent question — and the short answer is:

✅ Yes, for most generators, you can use .next() and .send(None) interchangeably — but there are important nuances you should know.

Let’s break it down clearly and precisely:

🔁

🔁 When next() and send(None) Are Equivalent

🔹 Basic generator:

```
def my_gen():
    yield "step 1"
    yield "step 2"

g = my_gen()

print(next(g))        # ✅ "step 1"
print(g.send(None))   # ✅ "step 2" — works like next()
```

✅ send(None) is exactly the same as next() when: • The generator is waiting at a bare yield • You don’t want to inject any actual data

✅

✅ When next() and send(None) Are Not Interchangeable

✅ Case 1: First call to a generator with yield = (expression)

```
def my_gen():
    name = yield "What's your name?"
    yield f"Hello, {name}"

g = my_gen()

print(next(g))         # ✅ primes the generator, yields "What's your name?"
print(g.send("Alice")) # ✅ sends "Alice" into the generator
```

✅ But this will fail:

```
g = my_gen()
g.send("Alice")  # ❌ TypeError: can't send non-None value to a just-started generator
```

You must prime the generator first using next() or send(None).

✅

✅ Case 2: When you want to inject a value into the generator

Then send(...) is the only option:

```
def echo():
    while True:
        val = yield
        print(f"Received: {val}")

g = echo()
next(g)         # ✅ primes to first `yield`
g.send("Hello") # ✅ sends "Hello" into the generator
```

Calling next(g) here again would just send None, which isn’t useful if you’re expecting real data.

✅

✅ Summary Table

Use Case	Use next()	Use send(None) Use send(value)
1. Prime a new generator	✅ Yes	✅ Yes ❌ No
2. Resume at a bare yield	✅ Yes	✅ Yes ⚠️ Only if value is None
3. Pass value to yield expression	❌ No	❌ No ✅ Yes

🧠 TL;DR • next() is shorthand for send(None) • You must use next() or send(None) to prime a generator (first step) • Use send(value) only after priming, when the generator is paused at name = yield • 🚫 Never use send(value) on a just-started generator — you'll get a TypeError

📊 Summary Chart

Task Type	Use `asyncio`?	Use Threads?	Notes
Network requests	✅ Yes	❌ Not needed unless using blocking libraries	
File I/O	❌ No	✅ Yes	Use `asyncio.to_thread()` or `ThreadPoolExecutor`
CPU-bound tasks	❌ No	✅ Use Process Pool	Use `multiprocessing` or `ProcessPoolExecutor`
Mixed I/O (files + network)	⚠️ with care	✅ use threads for files	Use async for network, threads for blocking ops

🧠 Final Recommendations

Scenario	Tool
Web scraping, API calls	`asyncio` + `aiohttp`
Read/write CSV, JSON, logs	`asyncio.to_thread()` or `ThreadPoolExecutor`
Heavy file parsing or computation	`ProcessPoolExecutor` (CPU-bound)
Want simplicity and flexibility	Use `unsync` to combine all styles

The Role of Each Synchronization Primitive

Think of these tools as different ways for coroutines to coordinate, not all of which involve passing data.

1. asyncio.Event (The Traffic Light) Purpose: Simple signaling. It's a binary flag that one coroutine can set (event.set()) to unblock one or more other coroutines that are waiting for it (await event.wait()). Data Passing: It does not pass data. When a coroutine is unblocked by an event, it simply resumes

execution. It doesn't receive any value. It's like a traffic light turning green—it tells you when to go, but doesn't hand you a package.

2. asyncio.Lock (The Key to a Room) Purpose: Mutual exclusion. It ensures that only one coroutine can enter a "critical section" of code at a time. Data Passing: It does not pass data directly. Instead, it protects a shared resource (like a list, dictionary, or object) that holds the data. The coroutines communicate by modifying this shared resource, and the lock just prevents them from doing it at the same time and causing corruption. The data is on the "whiteboard inside the room," not passed by the key itself.
3. asyncio.Future (The IOU) Purpose: Represents the eventual result of a single, one-time operation. Data Passing: It's a one-way, one-time data transfer. A producer coroutine can set a result on the future (future.set_result(value)), and a consumer can await it to get that single value. Once the result is set, the future is "done" and cannot be used to pass more data. It's like an IOU—you hand it over, and eventually, you get a single payment back. You can't use the same IOU for another payment. Why Queue and .send() Are Different asyncio.Queue (The Conveyor Belt): This is explicitly designed for streaming data. It's a channel where producers can continuously put items and consumers can continuously get them. It's the standard tool for decoupled, many-to-many communication in modern asyncio. Generator .send() (The Phone Call): This is the classic method for tightly coupled, two-way communication with a single generator-based coroutine. yield sends data out, and .send() sends data in. It is inherently "back and forth."

Summary Table

Primitive Purpose Data Flow Analogy asyncio.Queue Streaming data between tasks Many-to-many, continuous Conveyor Belt asyncio.Event Simple signaling None (just a signal) Traffic Light asyncio.Lock Protecting shared state Indirect (via the shared state) Key to a Room asyncio.Future A single eventual result One-way, one-time IOU / Promise Generator .send() Two-way communication One-to-one, back and forth Phone Call

To send data back and forth between multiple workers using async/await

1. asyncio.Queue
2. send(): generators can be used if we use @coroutine type for the async method

Returning data from a pipeline

That's an excellent and very practical question. You've correctly identified that you can't simply return a value from a long-running asynchronous function or pipeline without stopping it.

Practically this means from an async function running a while True loop, we cannot simply return a value, if we do so, it will exit the async task and data pipeline will be broken meaning the producer/consumer will stop

To get data out of a running system, you need to establish a communication channel back to the part of your program that needs the data.

create_task

Yes, asyncio.create_task() schedules the coroutine to start running on the event loop as soon as possible. It doesn't wait for you to await the task object.

Think of it this way:

Creating a Coroutine: my_coro = fetch_data() simply creates a coroutine object. It's like writing a recipe but not starting to cook. Creating a Task: task = asyncio.create_task(my_coro) is like handing that recipe to a chef (the event loop) and saying, "Start working on this as soon as you have a free moment." The chef will begin immediately, and if the recipe says "wait for 10 minutes," the chef won't just stand there. They will put the first dish in the oven and immediately start working on another recipe you've given them.

Awaiting the task (await task) is you waiting for the chef to tell you that specific dish is ready to serve. The cooking has been happening all along.

asyncio vs unsync - Thread Safety

1. When using pure asyncio, coroutines run within a single-threaded event loop, so it's safe to use asyncio.Queue, which is not thread-safe.
2. However, when using the unsync library, even @unsync async coroutines are executed in background threads (with their own event loops). This means multiple threads may access shared data like a queue.
3. Therefore, we use queue.Queue (which is thread-safe) instead of asyncio.Queue to safely share data between unsync-decorated coroutines and functions.

unsync future object

generate_data().result() call.

 What’s Happening in generate_data().result()?

1. @unsync Decorator

You’ve defined:

```
@unsync
async def generate_data(num: int, data: asyncio.Queue):
    for idx in range(1, num + 1):
        item = idx * idx
        # Use queue
        work = (item, datetime.datetime.now())
        # data.append(work)
        await data.put(work)

        print(colorama.Fore.YELLOW + " -- generated item {}".format(idx), flush=True)
        # Sleep better
        # time.sleep(random.random() + .5)
        await asyncio.sleep(random.random() + .5)
```

This means: • generate_data() returns an UnsyncFuture (not a coroutine). • It wraps the async function and runs it in a background thread with its own event loop.

2. .result() on UnsyncFuture

UnsyncFuture is a custom object returned by the @unsync decorator.

```
future = generate_data()
future.result()
```

This: • Blocks the main thread until the generate_data coroutine finishes. • Then returns the final result (if any). • Similar to calling .result() on a concurrent.futures.Future.

 Why Not Just await generate_data()?

Because: • generate_data() is not a coroutine, it’s an UnsyncFuture object. • You can’t await it unless you’re inside another async function and the coroutine was not decorated with @unsync.

 Summary

```
Code                          What It Does
---------------------------   ------------------------------------------------------------
-----------
1. @unsync async def f()         Wraps the async function to run in a background
thread

2. f()                           Returns an UnsyncFuture

3. f().result()                  Blocks until the async function completes and gets
the result

4. await f()                      Invalid – f is not an awaitable coroutine anymore
```

 Final Example in Plain Terms

```
@unsync
async def foo():
    await asyncio.sleep(1)
    return 42

future = foo()          # This runs foo in background thread
result = future.result()  # Wait for foo to finish and get the result (42)
```